

MONITOR Z · L06

Sockets en z/OS

Cuaderno de estudio · versión del 2026-10-05

Margen derecho y renglones libres para tus notas. Las láminas son las mismas de la carpeta **_visual**. Si una lección se corrige, se regenera este cuaderno con la fecha nueva.

L06 — Sockets en z/OS

Objetivo: que ZTKMON deje de escribir en SYSPRINT y **hable en vivo** con el puente de la PC por TCP. Al final sabés la secuencia de un servidor, por qué no hay que bloquearse sin plazo y cómo se arman las líneas que llegan partidas.

⚠ **Requiere de sistemas:** un **puerto TCP** reservado en DESA y una regla de firewall PC ↔ host. Sin eso, la teoría se estudia igual y la práctica se hace cuando estén.

1. Quién escucha y quién llama

En nuestro diseño **el host es el servidor**: ZTKMON escucha en un puerto. **La PC es el cliente**: el puente se conecta.

Decisión	Por qué
El host escucha	El puerto se reserva una vez en el host, y la PC puede reconectarse cuando quiera
Un cliente a la vez	Suficiente para la demo; simplifica el programa
El host rechaza IPs que no estén en su lista	Que no se conecte cualquiera de la red

2. La secuencia de un servidor TCP

z/OS ofrece una API de sockets para Assembler: la **macro EZASMI**. También existe la interfaz por `CALL` (EZASOKET), pero acá usamos la macro.

#	Llamada	Qué hace	Analogía 📞
1	INITAPI	Conecta el programa con TCP/IP	Contratar la línea
2	SOCKET	Crea un socket de tipo <i>stream</i> (TCP)	Comprar el aparato
3	BIND	Lo asocia al puerto	Asignarle el número
4	LISTEN	Lo pone a escuchar	Encender el timbre

5	ACCEPT	Atiende una conexión y devuelve otro socket para esa charla	Atender la llamada
6	READ / WRITE (o RECV / SEND)	Recibe y manda bytes	Hablar y escuchar
7	CLOSE	Cierra la charla (o el socket que escucha)	Colgar
8	TERMAPI	Se desconecta de TCP/IP	Dar de baja la línea

Cada llamada devuelve un código de retorno y un `ERRNO`. El `ERRNO` dice qué falló (puerto en uso, conexión cortada...) y se escribe siempre en el log del monitor.

 Fuente: z/OS Communications Server: IP Sockets Application Programming Interface Guide and Reference (macro EZASMI). Los nombres exactos de los parámetros se toman del manual al escribir P06.

Un servidor TCP: la secuencia de llamadas

L06 · Sockets en z/OS

#	Llamada (EZASMI)	Qué hace	Analogía del teléfono 📞
1	INITAPI	Conecta el programa con TCP/IP	Contratar la línea
2	SOCKET	Crea el socket (stream = TCP)	Comprar el aparato
3	BIND	Lo asocia a un puerto	Asignarle un número
4	LISTEN	Lo pone a escuchar conexiones	Encender el timbre
5	ACCEPT	Atiende una conexión y devuelve un socket nuevo para esa charla	Atender la llamada
6	READ / WRITE (o RECV / SEND)	Recibe y manda bytes	Hablar y escuchar
7	CLOSE	Cierra la charla (o el socket que escucha)	Colgar
8	TERMAPI	Se desconecta de TCP/IP	Dar de baja la línea

En nuestro diseño el host es el servidor (escucha en un puerto que asigna sistemas) y la PC es el cliente (se conecta). Además, el host rechaza IPs que no estén en su lista.

Fuente: z/OS Communications Server: IP Sockets Application Programming Interface Guide and Reference (EZASMI) · PROTOCOLO.md

Lámina L06-01_servidor_tcp

3. Bloqueo y plazos: la regla más importante

Por defecto, `ACCEPT` y `READ` son **bloqueantes**: la tarea queda esperando hasta que llegue algo.

- Si el puente no manda nada, el monitor **deja de vigilar** y no atiende un `P ZTKMON`.
- **Antecedente propio:** en ZTKAVISA (que es cliente), un `CONNECT` sin plazo tardó **77 s** en devolver el error.

Las dos salidas:

Opción	Cómo	Cuándo conviene
<code>SELECT</code> con timeout	Preguntar "¿hay algo para leer en estos sockets?" con un plazo; si no hay nada, se vuelve al bucle	La más simple de razonar
Modo asíncrono con ECB	La API postea un ECB cuando la operación termina; el ECB se suma a la <code>ECBLIST</code> de L02	Para que socket, timer y operador despierten al mismo <code>WAIT</code>

A confirmar en la práctica: cuál de las dos se usa y con qué parámetros exactos de EZASMI.

Bloqueo, plazos y por qué importa

L06 · Sockets en z/OS

❌ Llamada bloqueante sin plazo

`READ` espera **para siempre** si la PC no manda nada. Mientras tanto, el monitor **no vigila** y no atiende un `P ZTKMON`.

Antecedente: en ZTKAVISA (cliente) un `CONNECT` sin plazo tardó **77 s** en devolver el error.

✅ Esperar con plazo

Antes de leer, preguntar "¿hay algo para leer?" con `SELECT` y un **timeout**, o usar la forma asíncrona del API con un **ECB**. Así el socket se suma a la lista de cosas que despiertan al bucle (L02).

La variante exacta se elige y se prueba en la lección.

Tema	Qué tener en cuenta
Puerto	Lo reserva sistemas en el perfil de TCP/IP (sentencia <code>PORT</code>) para el STC
EBCDIC / ASCII	El host manda EBCDIC; la traducción a texto la hace el puente en la PC
Un cliente a la vez	Suficiente para la demo; el host rechaza IPs fuera de su lista
Mensajes partidos	TCP es un flujo : una línea puede llegar en dos pedazos. Se arma hasta encontrar <code>X'15'</code>

■ Analogía: esperar a alguien en la puerta

Bloqueante sin plazo es quedarse parado en la puerta hasta que llegue, aunque se queme la cocina. Con plazo es mirar por la ventana cada tanto y seguir con lo tuyo mientras tanto.

Fuente: EZASMI Guide and Reference (SELECT, modo asíncrono) · IDEA-005 (ZTKAVISA, CONNECT de 77 s) · PROTOCOLO.md

Lámina L06-02_bloqueo_y_plazos

4. TCP es un flujo, no mensajes

TCP entrega **bytes**, no líneas:

- Un `READ` puede traer **media línea**, una línea y media o tres líneas juntas.
- **Regla:** acumular en un buffer y cortar **en cada X'15'**. Lo que queda después del último X'15' espera al próximo `READ`.
- Al escribir, mandar la línea completa con su X'15'. Si `WRITE` dice que mandó menos bytes que los pedidos, mandar el resto.
- Del lado PC esto ya está resuelto: `punte.py` lee línea por línea y decodifica con `cp037`.

5. Qué hay que pedir y qué se configura

Tema	Detalle
Puerto	Lo reserva sistemas en el perfil de TCP/IP (sentencia <code>PORT</code>), asociado al STC
Firewall	Regla desde la IP de la PC hacia el puerto del host
Lista de IPs permitidas	La lleva el propio ZTKMON (PARM o miembro de configuración)
Translation	El host manda EBCDIC; la PC traduce. El host no convierte nada

6. Preguntas de comprobación

1. ¿Por qué `ACCEPT` devuelve un socket **nuevo** en lugar de usar el que escucha?
2. Llega un `READ` con `+EVT INI ZTKDOK JOB01` y nada más. ¿Qué hace el programa con esos bytes?
3. ¿Qué le pasa al monitor si hace `READ` sin plazo y el puente de la PC se cuelga?
4. ¿Por qué conviene que el socket también despierte el `WAIT ECBLIST`?
5. ¿Para qué sirve que el host tenga su propia lista de IPs permitidas, si ya hay un firewall?

Fuente: *IP Sockets API Guide and Reference* (EZASMI) · PROTOCOLO.md (transporte) · puente.py · IDEA-005 (ZTKAVISA, CONNECT de 77 s).

Notas